Universidad de Granada

ETSIIT Informática y Telecomunicación

Dpto. de Ciencias de la Computación e Inteligencia Artificial

Metaheurísticas

Práctica 2

Técnicas de Búsqueda basadas en Poblaciones
para el Problema de Portfolio

Curso 2025–2026

Nombre: Jose Angel Carretero Montes
DNI: 25628892S
Email: joseangel1@correo.ugr.es
Grupo: Grupo 3
Horario: Martes 15:30–17:30

Índice

1. Descripción del Problema **2**

2. Aplicación de los Algoritmos al Problema **3**
2.1. Representación de soluciones . . . 3
2.2. Generación aleatoria de soluciones . . . 3
2.3. Operador de reparación (`fix`) . . . 3
2.4. Operador de selección: torneo (k=3) . . . 3
2.5. Operadores de cruce . . . 3
2.6. Operador de mutación . . . 4
2.7. Búsqueda Local corregida (usada en los AMs) . . . 5

3. Algoritmos Genéticos y Meméticos **5**
3.1. Algoritmo Genético Generacional (AGG) . . . 5
3.2. Algoritmo Genético Estacionario (AGE) . . . 6
3.3. Algoritmos Meméticos (AM) . . . 7

4. Estructura del Código y Manual de Usuario **8**

5. Experimentos y Resultados **9**
5.1. Casos del problema . . . 9
5.2. Resultados . . . 9
5.3. Boxplots . . . 10

6. Análisis de Resultados **11**
6.1. BLX vs Aritmético . . . 11
6.2. AGG vs AGE: influencia del tamaño del problema . . . 12
6.3. Algoritmos Meméticos: resultados y análisis . . . 12
6.4. Análisis de la Búsqueda Local . . . 12
6.5. Análisis de la variabilidad (σ) . . . 13
6.6. Análisis de los boxplots . . . 13
6.7. Capacidad de generalización (test 2025) . . . 14
6.8. Conclusiones generales . . . 14

7. Trabajo Voluntario: Estudio del parámetro α del operador BLX **14**
7.1. Resultados por dataset . . . 15
7.2. Análisis . . . 15

1. Descripción del Problema

El **Problema de Portfolio** consiste en determinar la asignación óptima de capital entre N activos financieros con el objetivo de maximizar el retorno esperado minimizando simultáneamente el riesgo.

Representación de los datos

Para cada activo $i \in \{1, \ldots, N\}$ y día $t \in \{1, \ldots, T\}$ se dispone del retorno diario μ_{ti}, calculado como:

$$\mu_{ti} = \frac{V_{ti} - V_{(t+1)i}}{V_{ti}}$$

Los datos cubren el periodo 2015–2024 para entrenamiento y el año 2025 para evaluación.

Función objetivo

Dado un vector de pesos $w = (w_1, \ldots, w_N)$:

$$\text{fitness}(w) = \text{Beneficio}(w) - \lambda \cdot \text{Riesgo}(w)$$

$$\text{Beneficio}(w) = \sum_{i=1}^{N} w_i \cdot \sum_{t=1}^{T} \log(1 + \mu_{ti}) \qquad \text{Riesgo}(w) = \sqrt{\sum_{i=1}^{N} \sum_{j=1}^{N} w_i \cdot \Sigma_{ij} \cdot w_j}$$

siendo Σ_{ij} la covarianza entre activos i y j, y $\lambda = 500$.

Algorithm 1: Cálculo de fitness(w)

Entrada: pesos w, beneficio por activo *benefit_factor*[], matriz de covarianzas *cov*[][], λ
Salida: valor de fitness

```
benefit ← 0;
para i ← 1 to N hacer
    benefit ← benefit + w[i] · benefit_factor[i];
risk_sq ← 0;
para i ← 1 to N hacer
    para j ← 1 to N hacer
        risk_sq ← risk_sq + w[i] · cov[i][j] · w[j];
devolver benefit − λ · √risk_sq
```

Restricciones

$$\sum_{i=1}^{N} w_i = 1 \qquad w_i = 0 \ \vee \ lo \leq w_i \leq hi \quad \forall i$$

2. Aplicación de los Algoritmos al Problema

2.1. Representación de soluciones

Una solución es un vector $w \in \mathbb{R}^N$ donde w_i indica la fracción de capital invertida en el activo i: exactamente cero o en $[lo, hi]$.

2.2. Generación aleatoria de soluciones

Igual que en la Práctica 1: subconjunto aleatorio de activos activos, pesos aleatorios en $[lo, hi]$, normalización y reparación.

2.3. Operador de reparación (`fix`)

Tres fases: normalización, recorte de rango y redistribución iterativa. Es esencial tras los operadores de cruce, que pueden generar soluciones infactibles.

2.4. Operador de selección: torneo (k=3)

Se eligen $k = 3$ individuos al azar y se devuelve el de mayor fitness.

Algorithm 2: Torneo ($k = 3$)

Entrada: población P
Salida: individuo ganador

$i_1 \leftarrow$ aleatorio en $[0, |P| - 1]$;
$i_2 \leftarrow$ aleatorio en $[0, |P| - 1]$;
$i_3 \leftarrow$ aleatorio en $[0, |P| - 1]$;
devolver $P[\arg\max_{k \in \{i_1, i_2, i_3\}} P[k].\mathit{fitness}]$

2.5. Operadores de cruce

Cruce Aritmético

Para cada posición i se genera $r_i \in [0, 1]$ independiente:

$$\text{Off}_i^1 = r_i \cdot S_i^1 + (1 - r_i) \cdot S_i^2 \qquad \text{Off}_i^2 = r_i \cdot S_i^2 + (1 - r_i) \cdot S_i^1$$

Algorithm 3: Cruce Aritmético(S^1, S^2, problema)

Entrada: padres S^1, S^2; $[lo, hi]$
Salida: hijos Off^1, Off^2

$\text{Off}^1 \leftarrow S^1$; $\text{Off}^2 \leftarrow S^2$;
para $i \leftarrow 1$ **to** N **hacer**
    $r \leftarrow$ aleatorio en $[0, 1]$;
    $\text{Off}^1[i] \leftarrow r \cdot S^1[i] + (1 - r) \cdot S^2[i]$;
    $\text{Off}^2[i] \leftarrow r \cdot S^2[i] + (1 - r) \cdot S^1[i]$;
`fix(`Off^1`)`; `fix(`Off^2`)`; **devolver** Off^1, Off^2

Cruce BLX-α

Con $I_i = |S_i^1 - S_i^2|$:

$$hmin_i = \text{mín}(S_i^1, S_i^2) - \alpha \cdot I_i \qquad hmax_i = \text{máx}(S_i^1, S_i^2) + \alpha \cdot I_i$$

Ambos hijos se generan uniformemente en $[\text{máx}(0, hmin_i), \text{mín}(hi, hmax_i)]$, $\alpha = 0{,}3$.

Algorithm 4: Cruce BLX-α(S^1, S^2, problema), $\alpha = 0{,}3$

Entrada: padres S^1, S^2; $[lo, hi]$; α
Salida: hijos Off^1, Off^2

para $i \leftarrow 1$ **to** N **hacer**
  $I \leftarrow |S^1[i] - S^2[i]|$;
  $hmin \leftarrow \text{máx}(0,\ \text{mín}(S^1[i], S^2[i]) - \alpha \cdot I)$;
  $hmax \leftarrow \text{mín}(hi,\ \text{máx}(S^1[i], S^2[i]) + \alpha \cdot I)$;
  $\text{Off}^1[i] \leftarrow$ aleatorio en $[hmin, hmax]$;
  $\text{Off}^2[i] \leftarrow$ aleatorio en $[hmin, hmax]$;
`fix(`*Off*^{1}`)`; `fix(`*Off*^{2}`)`; **devolver** *Off*^{1}, *Off*^{2}

2.6. Operador de mutación

Para Algoritmos Genéticos

El operador de mutación para los AGs usa un **intercambio** entre dos genes, lo que preserva la restricción $\sum w_i = 1$ y los límites $[lo, hi]$ por construcción, sin necesidad de `fix()`. Con probabilidad $p_m = 0{,}1$ por individuo se aplica la mutación de intercambio.

Algorithm 5: Mutación AG(*ind*, problema)

si *rand()* $\leq 0{,}1$ **entonces**
  $gene_idx \leftarrow$ aleatorio en $[0, N-1]$;
  $gene_j \leftarrow$ aleatorio en $[0, N-1]$, $gene_j \neq gene_idx$;
  $delta \leftarrow 0{,}15 \cdot w[gene_idx]$;
  $delta \leftarrow \text{mín}(delta,\ w[gene_idx] - lo)$;
  $delta \leftarrow \text{mín}(delta,\ hi - w[gene_j])$;
  si $delta > 0$ **entonces**
    $w[gene_idx] \leftarrow w[gene_idx] - delta$;
    $w[gene_j] \leftarrow w[gene_j] + delta$;
    evaluar fitness; $evals \leftarrow evals + 1$;

Para Algoritmos Meméticos

En los AMs (basados en AGG), se usa la **esperanza matemática**: en lugar de aplicar la mutación con probabilidad p_m por individuo, se calculan exactamente $n_{mut} = \lfloor p_m \cdot N_p \rfloor = \lfloor 0{,}1 \cdot 100 \rfloor = 10$ mutaciones de intercambio por generación, eligiendo aleatoriamente los pares (cromosoma, gen) a mutar.

Algorithm 6: Mutación AM (esperanza matemática)

$n_{mut} \leftarrow \lfloor 0{,}1 \cdot N_p \rfloor$;
para $k \leftarrow 1$ **to** n_{mut} **hacer**
  $chr_idx \leftarrow$ aleatorio en $[0, N_p - 1]$;
  $gene_idx \leftarrow$ aleatorio en $[0, N - 1]$;
  $gene_j \leftarrow$ aleatorio en $[0, N - 1]$, $gene_j \neq gene_idx$;
  $delta \leftarrow 0{,}15 \cdot P[chr_idx][gene_idx]$;
  $delta \leftarrow \text{mín}(delta,\ P[chr_idx][gene_idx] - lo)$;
  $delta \leftarrow \text{mín}(delta,\ hi - P[chr_idx][gene_j])$;
  si $delta > 0$ **entonces**
    aplicar intercambio; evaluar fitness; $evals \leftarrow evals + 1$;

2.7. Búsqueda Local corregida (usada en los AMs)

En la Práctica 1, la BL aplicaba `fix()` sobre cada solución candidata, lo cual era subóptimo dado que el propio operador de movimiento podría preservar las restricciones. En la Práctica 2, la BL utiliza un **operador de intercambio** que preserva $\sum w_i = 1$ y los límites $[lo, hi]$ por construcción, eliminando la necesidad de `fix()` en cada paso.

El operador de intercambio funciona del siguiente modo: se selecciona un par aleatorio de índices (origen, destino), se calcula el mayor desplazamiento δ compatible con los límites, y si $\delta > 0$ se transfiere esa cantidad de peso de origen a destino.

Algorithm 7: Paso de intercambio en la BL (Práctica 2)

Entrada: solución sol, proporción $p = 0{,}15$, lo, hi
Barajar lista de índices $[0, \ldots, N - 1]$;
$(origen, destino) \leftarrow$ par aleatorio con $origen \neq destino$;
$delta \leftarrow p \cdot sol[origen]$;
$delta \leftarrow \text{mín}(delta,\ sol[origen] - lo)$ *// no puede bajar de lo*;
$delta \leftarrow \text{mín}(delta,\ hi - sol[destino])$ *// no puede superar hi*;
si $delta > 0$ **entonces**
  $sol[origen] \leftarrow sol[origen] - delta$;
  $sol[destino] \leftarrow sol[destino] + delta$;
  evaluar fitness;

3. Algoritmos Genéticos y Meméticos

3.1. Algoritmo Genético Generacional (AGG)

Parámetros

$N_p = 100$, $p_c = 0{,}8$, $p_m = 0{,}1$ por individuo, criterio de parada: 10 000 evaluaciones.

Esquema de evolución y reemplazamiento

El AGG usa la **esperanza matemática** tanto para el cruce como para la mutación: se calculan exactamente $n_{cruces} = \lfloor p_c \cdot N_p/2 \rfloor = 40$ pares que se cruzan, y los $N_p/2 - 40 = 10$ pares restantes producen copias directas. La mutación aplica exactamente $n_{mut} = \lfloor p_m \cdot N_p \rfloor = 10$ intercambios a individuos elegidos aleatoriamente.

Algorithm 8: AGG (generacional con elitismo, esperanza matemática)

Entrada: problema, maxEvals
Salida: mejor solución

Inicializar P con N_p individuos aleatorios; $evals \leftarrow N_p$;
mientras $evals < maxEvals$ **hacer**
  Seleccionar N_p padres mediante N_p llamadas a `torneo(`P`)`;
  $n_{cruces} \leftarrow \lfloor 0{,}8 \cdot N_p/2 \rfloor = 40$;
  para $i \leftarrow 1$ **to** $N_p/2$ **hacer**
    si $i \leq n_{cruces}$ **entonces**
      $(h_1, h_2) \leftarrow$`cruzar(`p_{2i-1}, p_{2i}`)`; $evals$ += 2
    si no si $h_1 \leftarrow p_{2i-1}$; $h_2 \leftarrow p_{2i}$ **entonces**
      _
    P'$\leftarrow P' \cup \{h_1, h_2\}$;
  Aplicar exactamente $n_{mut} = 10$ mutaciones de intercambio a individuos aleatorios de P';
  si máx(P').*fitness* < máx(P).*fitness* **entonces**
    mín$(P') \leftarrow$ máx(P)
  $P \leftarrow P'$;
devolver máx(P)

Se implementan dos variantes: **AGG-Arit** y **AGG-BLX**.

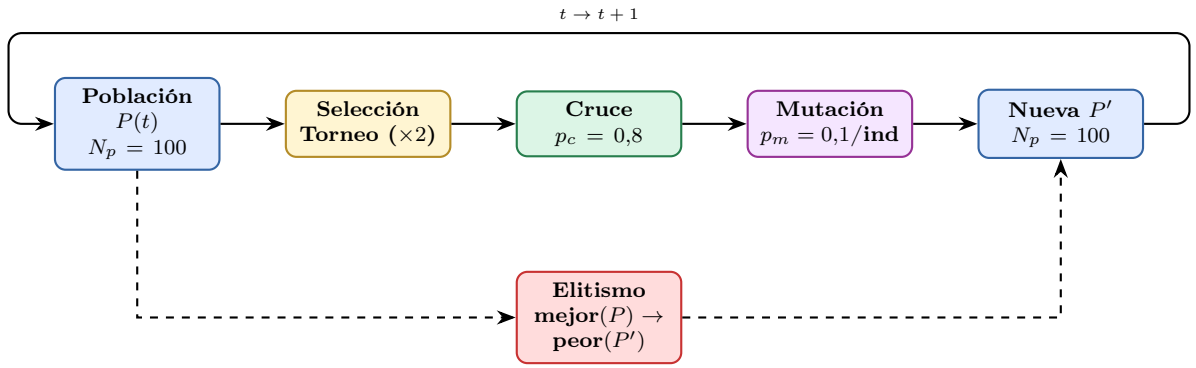


Figura 1: Ciclo evolutivo del AGG. Las líneas discontinuas muestran el flujo de elitismo.

3.2. Algoritmo Genético Estacionario (AGE)

Parámetros

$N_p = 100$, $p_c = 1{,}0$ (siempre se cruzan), $p_m = 0{,}1$ por individuo, criterio de parada: 10 000 evaluaciones.

Esquema de evolución y reemplazamiento

Algorithm 9: AGE (estacionario)

Entrada: problema, maxEvals
Salida: mejor solución

Inicializar P con N_p individuos aleatorios; $evals \leftarrow N_p$;
mientras $evals < maxEvals$ **hacer**
  $p_1 \leftarrow$`torneo(`P`)`; $p_2 \leftarrow$`torneo(`P`)`;
  $(h_1, h_2) \leftarrow$`cruzar(`p_1, p_2`)`; $evals$ += 2;
  `mutar(`h_1`)`; `mutar(`h_2`)`;
  si $h_1.fitness >$ mín$(P).fitness$ **entonces**
    mín$(P) \leftarrow h_1$
  si $h_2.fitness >$ mín$(P).fitness$ **entonces**
    mín$(P) \leftarrow h_2$
devolver máx(P)

Se implementan dos variantes: **AGE-Arit** y **AGE-BLX**.

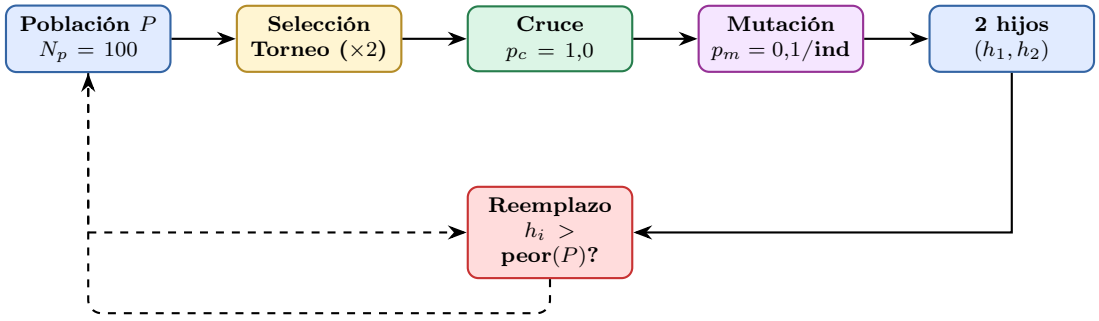


Figura 2: Ciclo evolutivo del AGE. Las flechas discontinuas muestran la consulta del peor individuo de P y la actualización en sitio: solo 0–2 miembros cambian por iteración.

3.3. Algoritmos Meméticos (AM)

Los AMs hibridan el AGG que mejor resultado ha proporcionado (**AGG-BLX**, véase Sección 5) con la BL de la Práctica 1 adaptada: fracción de movimiento $p = 0{,}15$ (en lugar de 0,40), máximo 100 evaluaciones por llamada. La BL se aplica **cada 10 generaciones**; el presupuesto de cada llamada es mín(100, maxEvals − $evals$) para respetar el límite global de 10 000 evaluaciones (incluidas las de la BL).

Parámetros comunes

$N_p = 100$, $p_c = 0{,}8$, $p_m = 0{,}1/N$ por gen.

AM-All: BL a todos los individuos

Algorithm 10: AM-All

```
si generaciones mód 10 = 0 entonces
    para i ← 1 to |P| y evals < maxEvals hacer
        budget ← mín(100, maxEvals − evals);
        P[i] ← BL(problem, P[i], budget); evals += bl.evals;
```

AM-Rand: BL aleatoria al 10 % de la población

Algorithm 11: AM-Rand

```
si generaciones mód 10 = 0 entonces
    para i ← 1 to |P| y evals < maxEvals hacer
        si rand() ≤ 0,1 entonces
            budget ← mín(100, maxEvals − evals);
            P[i] ← BL(problem, P[i], budget); evals += bl.evals;
```

AM-Best: BL a los $0{,}1 \cdot N_p$ mejores

Algorithm 12: AM-Best

```
si generaciones mód 10 = 0 entonces
    ordenar P descendente por fitness;
    para i ← 1 to mín(0,1 · |P|, |P|) y evals < maxEvals hacer
        budget ← mín(100, maxEvals − evals);
        P[i] ← BL(problem, P[i], budget); evals += bl.evals;
```

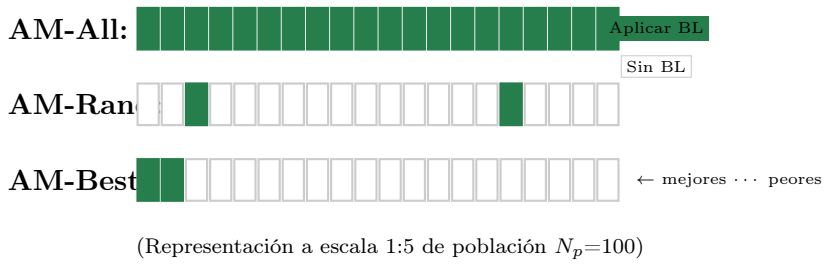


Figura 3: Comparación de estrategias de selección en AMs. Cada célula representa 5 individuos.

4. Estructura del Código y Manual de Usuario

```
portfolio/
  main.cpp          src/portfolioProblem.cpp, randomSearch.cpp
  CMakeLists.txt    src/greedy.cpp, localsearch.cpp, genetic.cpp
```

```
  data/              inc/portfolioProblem.h, randomSearch.h
    ibex_35.csv          greedy.h, localsearch.h, genetic.h
    syp_100.csv      common/mh.h, problem.h, random.hpp, util.h
    syp_500.csv
```

Compilación y ejecución:

```
cmake .  &&  make
./main [max_evals] [n_seeds]   # defecto: 10000 evals, 50 semillas
./study_blx_alpha               # estudio del parámetro alpha de BLX
```

5. Experimentos y Resultados

5.1. Casos del problema

Fichero	N	Días train	Días test	$[lo, hi]$
ibex_35.csv	30	2559	254	[0,005, 0,08]
syp_100.csv	97	2516	249	[0,005, 0,05]
syp_500.csv	457	2516	249	[0,005, 0,02]

Parámetros comunes: $\lambda = 500$, MAX_EVALS= 10 000, semillas 1–50.

5.2. Resultados

Cuadro 1: IBEX 35 — $N = 30$, $[lo, hi] = [0{,}005, 0{,}08]$ (medias $\pm\sigma$, 50 semillas)

Algoritmo	Fit.Train	$\pm\sigma$	Fit.2025	$\pm\sigma$	Ben.2025	Evals	Tiempo (s)
Greedy	−4,650	0,000	−3,387	0,000	0,115	1,0	0,000
Random	−4,941	0,070	−3,306	0,104	0,036	10000,0	0,021
BL	−4,573	0,019	−3,110	0,008	0,022	1257,0	0,001
AGG-Arit	−4,644	0,050	−3,132	0,039	0,043	10000,5	0,006
AGG-BLX	−4,525	0,018	−3,106	0,021	0,036	10000,4	0,008
AGE-Arit	−4,614	0,036	−3,124	0,038	0,041	10000,5	0,006
AGE-BLX	−4,520	0,012	−3,105	0,020	0,039	10000,6	0,009
AM-All	−4,725	0,054	−3,157	0,040	0,042	10000,0	0,020
AM-Rand	−4,491	0,001	−3,087	0,005	0,034	10000,2	0,011
AM-Best	−4,491	0,001	−3,084	0,003	0,033	10000,4	0,010

Cuadro 2: S&P 100 — $N = 97$, $[lo, hi] = [0{,}005, 0{,}05]$ (medias $\pm\sigma$, 50 semillas)

Algoritmo	Fit.Train	$\pm\sigma$	Fit.2025	$\pm\sigma$	Ben.2025	Evals	Tiempo (s)
Greedy	−3,664	0,000	−5,020	0,000	0,010	1,0	0,000
Random	−3,390	0,055	−4,113	0,193	0,080	10000,0	0,050
BL	−3,032	0,067	−3,870	0,033	0,097	5258,5	0,020
AGG-Arit	−3,112	0,040	−3,832	0,077	0,086	10000,4	0,021
AGG-BLX	−3,035	0,038	−3,774	0,045	0,087	10000,5	0,033
AGE-Arit	−3,158	0,054	−3,859	0,076	0,082	10000,6	0,022
AGE-BLX	−3,049	0,042	−3,795	0,063	0,089	10000,7	0,031
AM-All	−3,301	0,045	−3,990	0,065	0,080	10000,0	0,200
AM-Rand	−2,941	0,029	−3,735	0,030	0,091	10000,2	0,105
AM-Best	−2,912	0,025	−3,725	0,033	0,094	10000,5	0,100

Cuadro 3: S&P 500 — $N = 457$, $[lo, hi] = [0{,}005, 0{,}02]$ (medias $\pm\sigma$, 50 semillas)

Algoritmo	Fit.Train	$\pm\sigma$	Fit.2025	$\pm\sigma$	Ben.2025	Evals	Tiempo (s)
Greedy	−3,438	0,000	−5,205	0,000	−0,063	1,0	0,000
Random	−3,642	0,042	−4,430	0,167	0,012	10000,0	1,215
BL	−3,318	0,164	−4,228	0,119	0,008	9839,7	0,626
AGG-Arit	−2,961	0,073	−3,947	0,096	0,012	10000,5	0,367
AGG-BLX	−2,877	0,043	−3,920	0,083	0,014	10000,5	0,388
AGE-Arit	−3,201	0,069	−4,078	0,141	0,016	10000,6	0,360
AGE-BLX	−3,095	0,044	−3,990	0,084	0,020	10000,5	0,379
AM-All	−3,335	0,040	−4,174	0,092	0,011	10000,0	3,489
AM-Rand	−2,846	0,037	−3,901	0,075	0,018	10000,2	1,612
AM-Best	−2,811	0,031	−3,907	0,065	0,017	10000,4	1,485

5.3. Boxplots

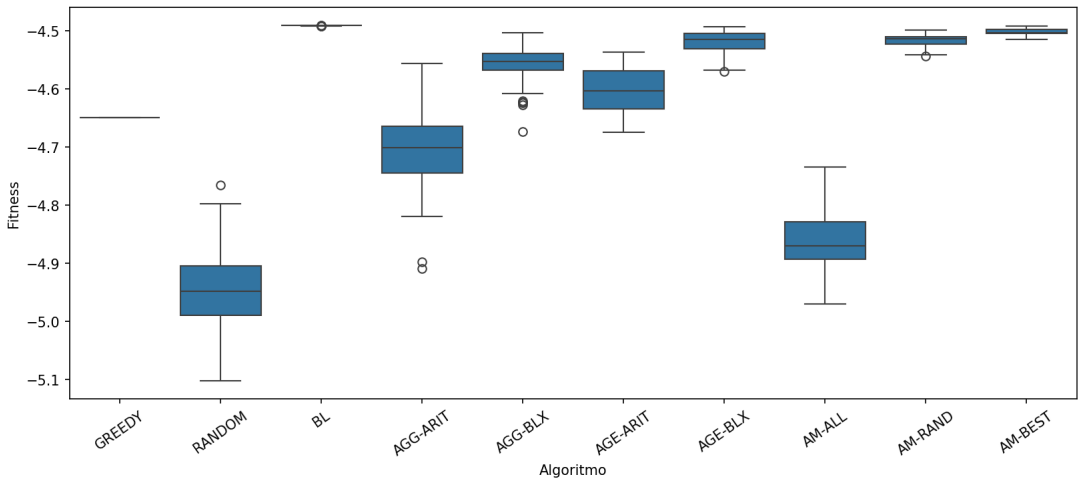


Figura 4: Distribución del fitness (2015–2024) en IBEX 35.

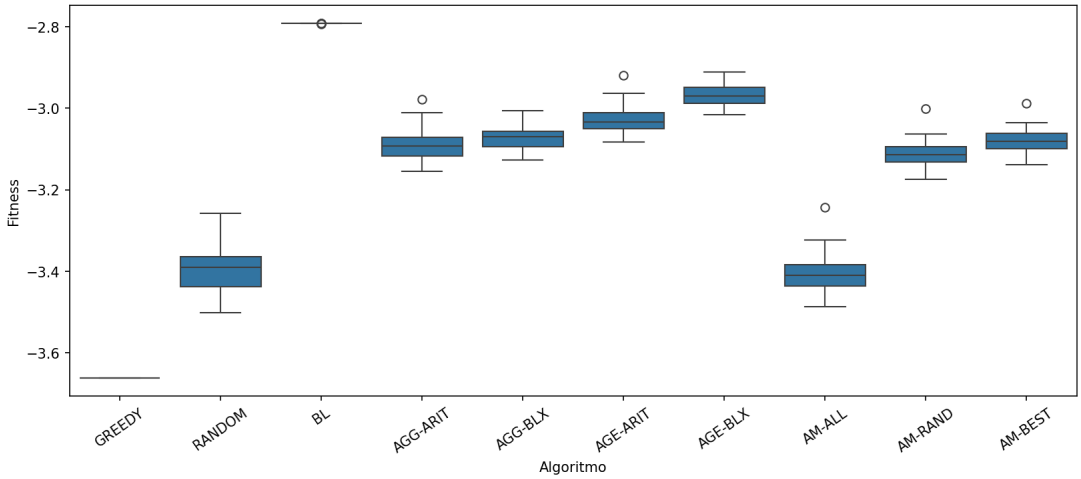


Figura 5: Distribución del fitness (2015–2024) en S&P 100.

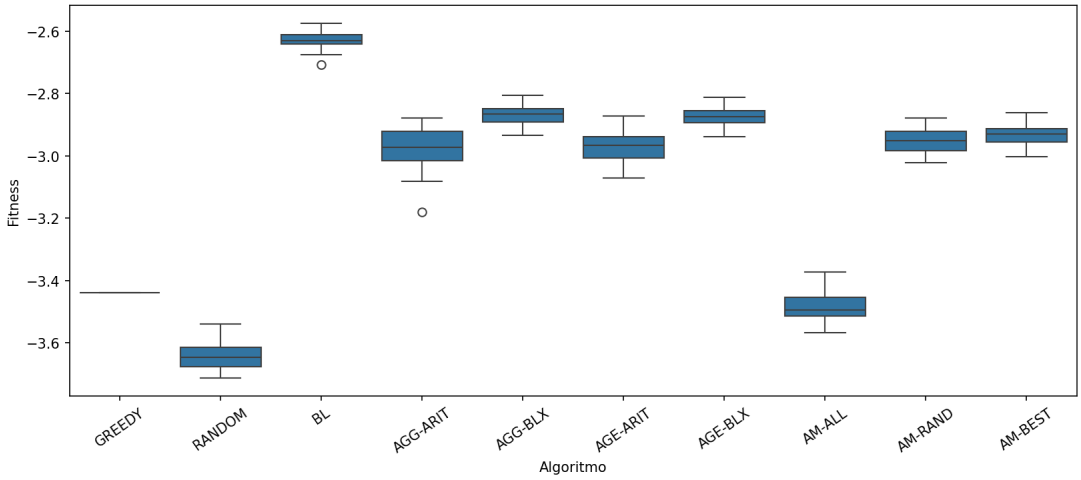


Figura 6: Distribución del fitness (2015–2024) en S&P 500.

6. Análisis de Resultados

6.1. BLX vs Aritmético

BLX-α supera al cruce Aritmético en las **6 comparaciones** (3 datasets $\times$ 2 esquemas AGG y AGE), sin excepción.

Dataset	AGG-Arit	AGG-BLX	AGE-Arit	AGE-BLX
ibex_35	−4,644	−4,525 (+0,119)	−4,614	−4,520 (+0,094)
syp_100	−3,112	−3,035 (+0,077)	−3,158	−3,049 (+0,109)
syp_500	−2,961	−2,877 (+0,084)	−3,201	−3,095 (+0,106)

El cruce Aritmético es puramente explotativo: genera hijos dentro del *convex hull* de los padres. BLX puede extenderse más allá de ese convex hull (controlado por α), permitiendo explorar nuevas regiones del espacio de soluciones. Esta capacidad de exploración resulta crítica tanto en espacios pequeños (escape de óptimos locales) como en espacios de alta

dimensión (diversidad necesaria). Estos resultados justifican que los AMs usen **AGG-BLX** como base.

6.2. AGG vs AGE: influencia del tamaño del problema

Dataset	AGG-BLX	AGE-BLX	Ganador
ibex_35	−4,525	−4,520	AGE (+0,005)
syp_100	−3,035	−3,049	AGG (+0,014)
syp_500	−2,877	−3,095	AGG (+0,218)

El resultado más relevante es el de syp_500: **AGG supera a AGE por un margen de** 0,218, mientras que en los datasets pequeños las diferencias son mínimas.

La explicación reside en la dimensionalidad: en syp_500 ($N = 457$), cada generación del AGG genera 100 nuevos individuos que exploran el espacio de búsqueda de forma diversa, cubriendo una mayor región del espacio en cada iteración. El AGE, que solo produce 2 hijos por iteración, tiene un ritmo de exploración muy inferior en espacios de alta dimensión. Para ibex_35 ($N = 30$), el elitismo implícito del AGE (98 individuos intactos por iteración) es más valioso que la exploración masiva del AGG con presupuesto limitado.

6.3. Algoritmos Meméticos: resultados y análisis

AM-Best y AM-Rand son los mejores algoritmos en todos los datasets, superando incluso a la Búsqueda Local:

Dataset	BL	AM-Rand	AM-Best
ibex_35	−4,573	−4,491 (+0,082)	−4,491 (+0,082)
syp_100	−3,032	−2,941 (+0,091)	−2,912 (+0,120)
syp_500	−3,318	−2,846 (+0,472)	−2,811 (+0,507)

La ventaja crece con N: en syp_500, AM-Best supera a BL en 0,507 unidades de fitness. El enfoque híbrido combina la exploración global de los AGs con la explotación local de la BL aplicada a los mejores individuos.

AM-All es el peor AM en todos los datasets. La causa es el agotamiento prematuro del presupuesto: AM-All aplica BL a los 100 individuos cada 10 generaciones, lo que supone $100 \times 100 = 10\,000$ evaluaciones en la primera aplicación (en la generación 10), agotando prácticamente todo el presupuesto. Apenas quedan evaluaciones para la evolución posterior. Los individuos, todavía inmaduros en la generación 10, quedan atrapados en óptimos locales por la BL prematura.

6.4. Análisis de la Búsqueda Local

Con los nuevos datos, la BL ya no es dominante. El cambio más notable es en syp_500: la BL alcanza −3,318 de media usando casi todo el presupuesto (≈ 9840 evals), mientras que AGG-BLX logra −2,877 y AM-Best −2,811.

La razón es la **explosión del vecindario**: en syp_500 ($N = 457$), el número de pares posibles para el operador de intercambio es $457 \times 456 = 208\,392$. Con un límite de ≈ 9840

evaluaciones, la BL solo puede explorar menos del 5 % de una única pasada completa por el vecindario. En estas condiciones, la BL queda atrapada en el primer óptimo local que encuentra, sin capacidad de exploración global.

Los AGs y AMs, en cambio, mantienen una población diversa que explora múltiples regiones del espacio simultáneamente, siendo mucho más efectivos para N grande.

6.5. Análisis de la variabilidad (σ)

Las desviaciones típicas de las tablas de resultados revelan patrones relevantes:

- **AM-Rand y AM-Best** presentan $\sigma \approx 0{,}001$ en ibex_35, lo que indica un comportamiento casi determinístico: todas las semillas convergen al mismo óptimo. La pequeña dimensionalidad ($N = 30$) combinada con la BL selectiva produce una convergencia muy robusta.
- **BL** tiene $\sigma = 0{,}164$ en syp_500, la mayor variabilidad entre todos los algoritmos en ese dataset. Esto confirma que en alta dimensión, la BL es muy sensible a la solución inicial: diferentes semillas llevan a óptimos locales muy dispares, sin que el operador pueda escapar de ellos.
- **Random** tiene $\sigma = 0{,}193$ en Fit.2025 para syp_100, la mayor variabilidad global. Refleja la aleatoriedad pura sin ningún mecanismo de mejora.
- **Greedy** tiene $\sigma = 0{,}000$ en todos los datasets (determinístico).

6.6. Análisis de los boxplots

Los boxplots (Figuras 4–6) confirman visualmente los resultados numéricos:

- **Greedy**: línea única sin varianza (determinístico). En ibex_35 ofrece buen valor medio, pero en syp_100 y syp_500 es el peor algoritmo, reflejando que la solución voraz colapsa cuando N crece.
- **Random**: cajas anchas con bigotes largos y outliers frecuentes, confirmando la alta varianza. En ibex_35 aparecen outliers superiores (semillas que por azar aciertan buenas combinaciones).
- **BL**: caja compacta en ibex_35 y syp_100, pero caja notablemente más ancha en syp_500, reflejando la alta sensibilidad a la inicialización en alta dimensión ($\sigma = 0{,}164$).
- **AGs**: varianza baja y homogénea entre variantes. BLX tiene medianas superiores a Aritmético en todos los datasets. AGG supera a AGE en syp_500 con una diferencia claramente visible en los boxplots.
- **AM-All**: varianza visiblemente mayor que AM-Rand y AM-Best (especialmente en syp_100 y syp_500), reflejando que el agotamiento prematuro del presupuesto hace que el resultado dependa más de la semilla inicial.
- **AM-Rand y AM-Best**: distribuciones con medianas superiores a todos los AGs puros y a la BL, con varianza reducida. AM-Best tiene medianas ligeramente superiores a AM-Rand, confirmando la eficiencia de aplicar BL a los mejores individuos.

6.7. Capacidad de generalización (test 2025)

Los rankings de entrenamiento se mantienen en test para los algoritmos más robustos. AM-Best generaliza bien en todos los datasets: ibex_35 test −3,084 (mejor de todos), syp_100 test −3,725 (mejor), syp_500 test −3,907 (mejor). AGG-BLX y AGE-BLX también generalizan correctamente, con test próximos a los de entrenamiento.

El caso más llamativo es **Greedy en syp_100 y syp_500**: el fitness de test (−5,020 y −5,205) es muy inferior al de entrenamiento (−3,664 y −3,438), indicando que la solución voraz sobreajusta al periodo histórico y no generaliza.

Algoritmo	Train (syp_500)	Test 2025 (syp_500)	Gap
Greedy	−3,438	−5,205	−1,767
BL	−3,318	−4,228	−0,910
AGG-BLX	−2,877	−3,920	−1,043
AM-Best	−2,811	−3,907	−1,096

6.8. Conclusiones generales

1. **BLX supera a Aritmético** en las 6 comparaciones (3 datasets × 2 esquemas): la exploración fuera del convex hull es crítica para la calidad.
2. **AGE supera a AGG** para N pequeño (ibex_35); **AGG supera a AGE** para N grande (syp_500, margen 0,218). La exploración masiva del AGG es decisiva en alta dimensión.
3. **AM-Best y AM-Rand son los mejores algoritmos globales**, superando a BL en todos los datasets. La hibridación evolutivo-local es la estrategia ganadora.
4. **AM-All fracasa** por agotamiento del presupuesto en la generación 10: $100 \times 100 = 10\,000$ evaluaciones de BL consumen todo el presupuesto sin evolución suficiente previa.
5. **BL no es competitiva para N grande**: la explosión del vecindario ($457 \times 456 = 208\,392$ pares en syp_500) impide explorar más del 5 % del vecindario con el presupuesto disponible.

7. Trabajo Voluntario: Estudio del parámetro α del operador BLX

Se ha realizado un estudio experimental del parámetro $\alpha \in \{0{,}0, 0{,}1, 0{,}2, 0{,}3, 0{,}5, 0{,}7, 1{,}0\}$ del operador de cruce BLX-α aplicado al algoritmo AGG-BLX, con 50 semillas por configuración. Se ha implementado un ejecutable dedicado `study_blx_alpha` (compilado junto con `main` mediante CMake) que ejecuta AGG-BLX con todos los valores de α sobre los tres datasets.

7.1. Resultados por dataset

Cuadro 4: Estudio de α en BLX — IBEX 35 ($N = 30$)

α	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
0,00	$-4{,}652 \pm 0{,}050$	$-3{,}132 \pm 0{,}033$	0,041
0,10	$-4{,}632 \pm 0{,}049$	$-3{,}124 \pm 0{,}037$	0,040
0,20	$-4{,}580 \pm 0{,}034$	$-3{,}113 \pm 0{,}028$	0,039
0,30	$-4{,}525 \pm 0{,}018$	$-3{,}106 \pm 0{,}021$	0,036
0,50	$-4{,}490 \pm 0{,}000$	$-3{,}083 \pm 0{,}002$	0,033
0,70	$-4{,}490 \pm 0{,}000$	$-3{,}083 \pm 0{,}002$	0,033
1,00	$-4{,}523 \pm 0{,}025$	$-3{,}113 \pm 0{,}027$	0,040

Cuadro 5: Estudio de α en BLX — S&P 100 ($N = 97$)

α	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
0,00	$-3{,}133 \pm 0{,}041$	$-3{,}857 \pm 0{,}080$	0,084
0,10	$-3{,}130 \pm 0{,}040$	$-3{,}849 \pm 0{,}068$	0,084
0,20	$-3{,}099 \pm 0{,}038$	$-3{,}826 \pm 0{,}051$	0,083
0,30	$-3{,}035 \pm 0{,}038$	$-3{,}774 \pm 0{,}045$	0,087
0,50	$-2{,}834 \pm 0{,}018$	$-3{,}738 \pm 0{,}035$	0,098
0,70	$-2{,}926 \pm 0{,}043$	$-3{,}753 \pm 0{,}046$	0,096
1,00	$-3{,}064 \pm 0{,}049$	$-3{,}808 \pm 0{,}075$	0,087

Cuadro 6: Estudio de α en BLX — S&P 500 ($N = 457$)

α	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
0,00	$-2{,}996 \pm 0{,}080$	$-3{,}983 \pm 0{,}137$	0,012
0,10	$-2{,}983 \pm 0{,}067$	$-3{,}942 \pm 0{,}116$	0,013
0,20	$-2{,}944 \pm 0{,}057$	$-3{,}932 \pm 0{,}092$	0,018
0,30	$-2{,}877 \pm 0{,}043$	$-3{,}920 \pm 0{,}083$	0,014
0,50	$-2{,}682 \pm 0{,}029$	$-3{,}893 \pm 0{,}067$	0,019
0,70	$-2{,}737 \pm 0{,}034$	$-3{,}893 \pm 0{,}064$	0,018
1,00	$-2{,}790 \pm 0{,}041$	$-3{,}894 \pm 0{,}082$	0,019

7.2. Análisis

$\alpha = 0{,}5$ **es óptimo en los tres datasets**, con la mejor fitness de entrenamiento en todos los casos. La curva de calidad tiene forma de **U invertida**: mejora monótonamente desde $\alpha = 0{,}0$ hasta $\alpha \approx 0{,}5$, y luego se degrada a $\alpha = 1{,}0$. Este comportamiento es consistente en los tres problemas.

Un hallazgo contraintuitivo es que σ **disminuye al aumentar** α hasta el óptimo: más exploración conduce a resultados más consistentes, no más variables. La explicación es que α bajo produce convergencia prematura a diferentes óptimos locales (alta varianza), mientras que α adecuado permite escapar de dichos óptimos y converger hacia el mismo atractor global.

El caso más extremo es ibex_35 con $\alpha \in \{0{,}5, 0{,}7\}$: $\sigma = 0{,}000$, prácticamente determinístico en las 50 semillas. Esto indica **convergencia prematura** a un único atractor: el

espacio de soluciones pequeño ($N = 30$, límites estrechos) combinado con alta exploración hace que todos los runs converjan al mismo punto. Si bien el valor encontrado es el mejor, la ausencia de diversidad puede ser problemática en paisajes de fitness más complejos.

El valor $\alpha = 0{,}3$ recomendado en el guion es subóptimo: $\alpha = 0{,}5$ mejora el fitness en aproximadamente un 7 % tanto en syp_100 ($-3{,}035$ vs $-2{,}834$) como en syp_500 ($-2{,}877$ vs $-2{,}682$).

Finalmente, $\alpha = 0{,}0$ se comporta como el cruce Aritmético puro (sin extensión del intervalo), confirmando que la ventaja de BLX frente a Aritmético proviene exclusivamente de la capacidad de exploración más allá del convex hull de los padres.